PRAGADESH KARTHICK

.NET Software Engineer

Thanjavur, Tamil Nadu, India • 6385295331 (primary) • pragadesh2408@gmail.com

LinkedIn: linkedin.com/in/pragadesh-karthick

PROFESSIONAL SUMMARY

.NET Software Engineer with 5+ years of experience building enterprise applications across the full development lifecycle, using C#, Blazor, EF Core, and SQL Server. Focused on optimizing query performance through improved filtering, missing-index analysis, and included columns, and on scenario-based implementation validated with xUnit and FluentValidation to minimize bugs and QA/UAT bounce-backs. Experienced working within JWT-based authentication systems and existing middleware frameworks. Gaining hands-on experience with AI agentic tools (Claude) to accelerate implementation and development cycles.

TECHNICAL SKILLS

Languages & Frameworks: C#, ASP.NET MVC, ASP.NET Core, .NET Core, Blazor Server, ADO.NET

Architecture & Patterns: Onion Architecture, MVVM, Repository Pattern, Unit of Work (UoW),CQRS

Data & ORM: Entity Framework, EF Core, LINQ (incl. dynamic LINQ), SQL Server, Stored Procedures (incl. dynamic SP), Query Performance Optimization

Security & Auth : JWT Bearer Tokens

API & Real-Time : Swagger , API Versioning (URL segment), AutoMapper**.**Postman

Validation & Testing : FluentValidation (config-driven dynamic rules), xUnit framework (Arrange ,Act,Assertion)

UI Libraries: Syncfusion for blazor ,DevExpress for Active Server Page

Tools & Practices: Azure Repos, Azure Boards, Git, Agile/Scrum

AI-Assisted Development : Claude Code (VS Code integration), prompt engineering for debugging, code review, and refactoring; Workflows, preparing qa testing scenarios.

PROFESSIONAL EXPERIENCE

Software Engineer — IntechHub Solution

Jun 2023 – Present

Main Projects : Claim Review Management System (CRMS)

Project Details

Claim Review Management System (CRMS) is a **multi-tenant SaaS platform** that enables insurance companies to manage the complete claim review lifecycle, including claim creation, multi-stage doctor reviews, re-appeals, dashboards, search, and reporting. The application was developed using **Blazor Server, ASP.NET Core 6.0, EF Core 6.0, and SQL Server**.

My Contributions

- Developed and enhanced core modules including **claim creation, multi-stage review workflows, dashboards, and multi-filter claim search**.
- Developed **dynamic LINQ queries** using a code-first approach and dynamic SQL stored procedures to support varying claim scenarios.
- Worked with the existing **JWT authentication and middleware framework**, enhancing exception handling and extending middleware functionality.
- Implemented **role-based and policy-based authorization** for secure access control.
- Implemented **rule-based validation using FluentValidation**, with configuration-driven validation rules.
- Improved application performance using **async programming, database indexing, pagination, SQL query optimization, and code refactoring**.
- Implemented and maintained business logic validation using **xUnit** unit testing.
- Developed UI components using **Syncfusion** within the Blazor Server application.
- Used **Claude AI** as a development assistant to accelerate implementation and debugging.
- Collaborated with **Team Leads and onsite teams** for requirement analysis, sprint delivery, issue resolution, and production support.

Sub Projects : CRMS RESTful API

Project Details

CRMS API is a RESTful API developed to create and process claims from plain-value claim requests. The API exposes Swagger documentation and uses token-based authentication for secure access. Incoming claim data is first stored in landing tables, where the plain values are resolved against the actual data source values before being transformed and loaded into the actual claim tables using an ETL-based approach.

My Contributions

- Developed the **CRMS RESTful API** for receiving and processing claim creation requests.
- Implemented **Swagger/OpenAPI** documentation for API testing and integration.
- Implemented **token-based authentication** to secure API endpoints.
- Developed claim processing logic to receive **plain-value claim data** through API requests.
- Implemented **Member Resolver and Value Resolver** logic to convert incoming plain values into their corresponding actual data-source values.
- Designed the data processing flow using an **ETL approach**:
  Landing → Transform → Actual Claim Tables.
- Implemented the data resolution and transformation process between the **Landing and Transform layers**.
- Used the **CQRS pattern** to separate claim command/write operations from other application responsibilities.
- Implemented validation and processing logic to ensure claim data was correctly transformed before being loaded into the actual claim tables.

Sub Projects : Pronto UI Application

Project Description

The project was a dental claim image-processing system that automated claim processing using PGP-encrypted files. Claims were extracted based on configured dental tooth numbers, associated images were retrieved using Image Source IDs, processed through an external Python library, and submitted to an AI system through FTP. AI responses were captured through file monitoring, correlated with the claim, and archived.

My Contributions

- Implemented PGP decryption to securely process incoming claim files.
- Developed claim extraction and filtering based on configured dental tooth numbers.
- Implemented Image Source ID handling and retrieval of images from external third-party sources.
- Implemented image download and local repository management.
- Integrated the external Python image-processing library and passed required images for processing.
- Implemented FTP-based transfer of processed images to the AI-processing system.
- Developed file-watching logic to capture AI response files and extract response codes.
- Implemented claim retry logic, allowing failed claims to be reprocessed within the same day.
- Enhanced the system to process multiple clients' claim data simultaneously.
- Implemented correlation between Image Source IDs, claims, and AI response codes.
- Implemented archival of processed files with claim and date-based organization.
- Implemented PGP encryption of final response files and placed them in the required destination folder.

.NET Developer — K.R.A Systems *Mar 2021 – May 2023*

Progressed from beginner to .NET Developer, building strong foundations in C#, ASP.NET MVC, ADO.NET, and SQL Server across the full development lifecycle.

Orion Fleet Intelligence

Compliance and safety platform for fleet operators — helping clients meet regulatory requirements, track vehicles in real time, and reduce operational risk across their vehicle fleets.

- Implemented HERE Maps integration for real-time fleet tracking, improving location accuracy
- Developed Vehicle Location Tracking and Passenger Demand Improvement modules using C#, DevExpress, and SQL Server
- Delivered client-side and server-side changes; maintained Windows Services and database components
- Contributed to compliance features that reduced operational risk and manual compliance-check effort

EDUCATION

B.E. in Computer Science & Engineering , Anjalai Ammal Mahalingam Engineering College - 2016 – 2020 | CGPA 6.9/10

PERSONAL PROJECTS

QueryOptimizer — SQL Query Optimizer with Verified Benchmarking

Personal portfolio project built independently of employer work. Benchmarks SQL query performance and validates AI-suggested query rewrites against real execution statistics — logical reads, CPU/elapsed time, and execution plan — rather than trusting AI output directly. Built using Clean Architecture, CQRS (MediatR), EF Core, and SQL Server.

GitHub: github.com/PragadeshK-DotNetDEV/QueryBench

ObjectTracker — Real-Time Distributed Tracing Dashboard

Personal portfolio project that automatically tracks an object's journey across application layers with minimal instrumentation, capturing state snapshots at each step and streaming live updates to a real-time visual dashboard with bottleneck detection.

GitHub: github.com/PragadeshK-DotNetDEV/ObjectTracker